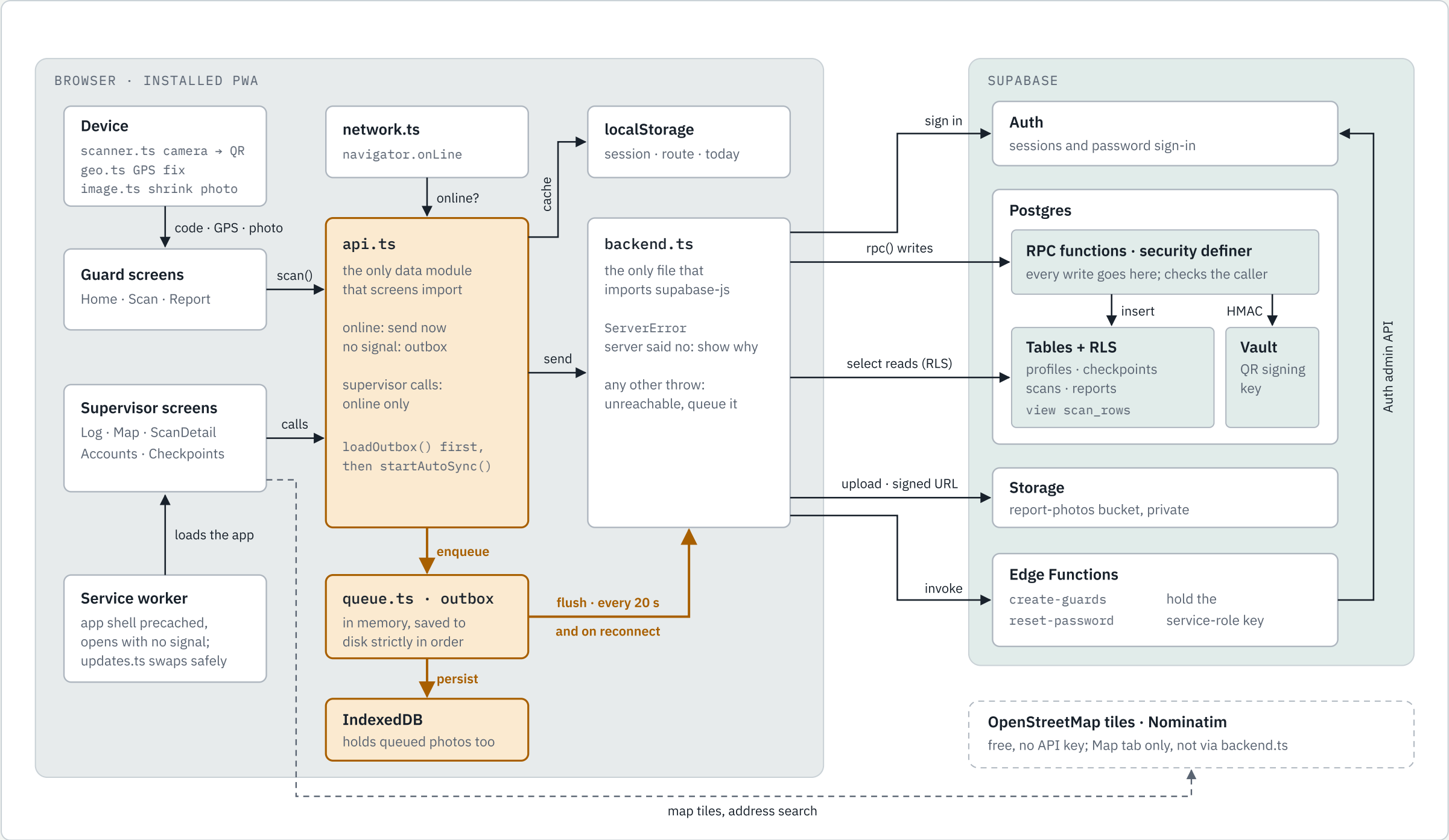


# Patroli System Map

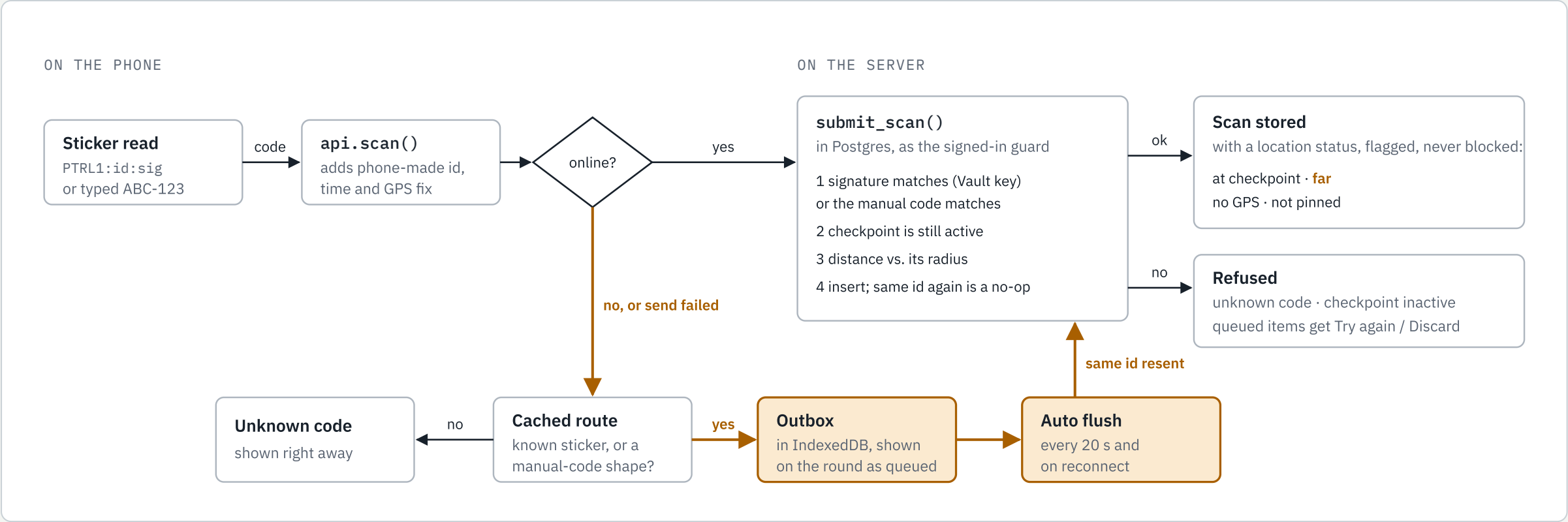
Guards scan QR stickers on their round, often with no signal. Supervisors read the log at a desk. The code splits along those two situations: the guard's phone can work alone for hours, and the server decides everything that matters.

## The whole system



Screens never touch Supabase. They call `api.ts`, which either sends through `backend.ts` or parks the work in the outbox. ■ Amber marks the offline path. On the server, reads go straight to tables under Row Level Security, and every write goes through a function that checks who is calling.

# What happens to one scan



The id is made on the phone, so sending the same scan twice stores it once. The phone can only guess offline. Only the server checks the QR signature, and the scan keeps the phone's scannedAt time next to the server's receivedAt.

## Rules the design depends on

### Two import boundaries

Screens import only `data/api.ts`. Only `data/backend.ts` imports `supabase-js`. You can swap the backend by changing one file.

### The guard app starts offline

`main.tsx` loads the outbox before the first render. The route, session and today's scans are cached in `localStorage`. The service worker serves the app shell.

### No direct table writes

RLS lets guards read their own rows and supervisors read everything. Every insert or update runs inside a `security definer` function that checks the caller's role.

### Updates never interrupt a scan

`updates.ts` reloads into a new version only on a screen with nothing to type, and only after the outbox has been saved.

### Stickers carry a signature

The QR holds `PTRL1:id:HMAC(id, qr_version)`, signed with a key kept in Vault. "Replace sticker" bumps `qr_version`, so copies of the old sticker stop working.

### Admin keys stay server-side

Creating accounts and resetting passwords need the service-role key. Only the two Edge Functions hold it, and each checks for an active supervisor first.

# Where to look

| CONCERN                | FILES                                                                                                                             |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Routing and role gates | <code>src/app.tsx</code> (hash routes, <code>RequireRole</code> ), <code>src/state.tsx</code> (user, language)                    |
| Send now or queue      | <code>src/data/api.ts</code> : <code>scan</code> , <code>addReport</code> , <code>flushOutbox</code> , <code>todayProgress</code> |
| Offline storage        | <code>src/data/queue.ts</code> (IndexedDB via <code>idb-keyval</code> , falls back to <code>localStorage</code> )                 |
| Supabase calls         | <code>src/data/backend.ts</code> , <code>src/supabaseClient.ts</code> , <code>database.types.ts</code>                            |
| Camera, GPS, photos    | <code>src/lib/scanner.ts</code> , <code>src/lib/geo.ts</code> , <code>src/lib/image.ts</code>                                     |
| Schema, RLS, functions | <code>supabase/migrations/</code> , starting with <code>20260923120000_patrol_schema.sql</code>                                   |
| Account admin          | <code>supabase/functions/create-guards</code> , <code>reset-password</code> , <code>_shared/supervisor.ts</code>                  |
| PWA and updates        | <code>vite.config.ts</code> ( <code>vite-plugin-pwa</code> ), <code>src/lib/updates.ts</code>                                     |